

Chapter 11 需求模型：行为,模式和Web/Mobile应用Behavior, Pattern and Web/Mobile App

sequence diagram图的作用：

1. V和类图互相验证
2. C为类的协作提供依据；
3. S和swim lane diagram互相印证（类在所有的sequence diagram都要出现）

11.1 - 11.4 行为模型（专注后端）

11.5 Web&Mobile模型（专注前端）

状态图：对类方法设计的重要参考依据

行为建模描述了系统的状态及其Class以及事件对这些状态的影响

11.1 行为模型建立 Behavioral Model

【以use case为基础-->找到事件-->构建sequence-->stae-->reivew】

建立步骤：

1. CREATING A BEHAVIORAL MODEL 评估所有的**UseCase**以完全了解系统交互的sequence **11.1**
2. IDENTIFYING EVENTS WITH THE USE CASE 认清驱动交互sequence的所有**Events**以及它们相关的**Objects** **11.3**
3. STATE REPRESENTATIONS I 为系统或各种类画状态图**StateDiagram** **11.4**
4. STATE REPRESENTATIONS II 对每个用例图画时序图**SequenceDiagram** **11.4**
5. 检查**Examine**行为模型验证准确性与一致性

两种图：状态图，时序图

11.2 认清Use Case的事件 Events

需要做的：

- 对每个事件events确定actor行动者
- 记录要交换的信息
- 罗列所有约束条件

所有事件确认后：将事件分配到对象中

对象可以负责生成事件；识别在其他地方发生的事件

11.3 状态展示 State Presentations

在行为建模的上下文中，必须考虑两种不同的状态特征：

- (1) 当系统执行其功能时，每个类的状态 **类的状态**
- (2) 当系统执行其功能时从外部观察到的系统状态。 **系统状态**

primary result of the password

① 找事件 ($e_1, e_2, \dots, e_n$)

② 状态流

deling, two different

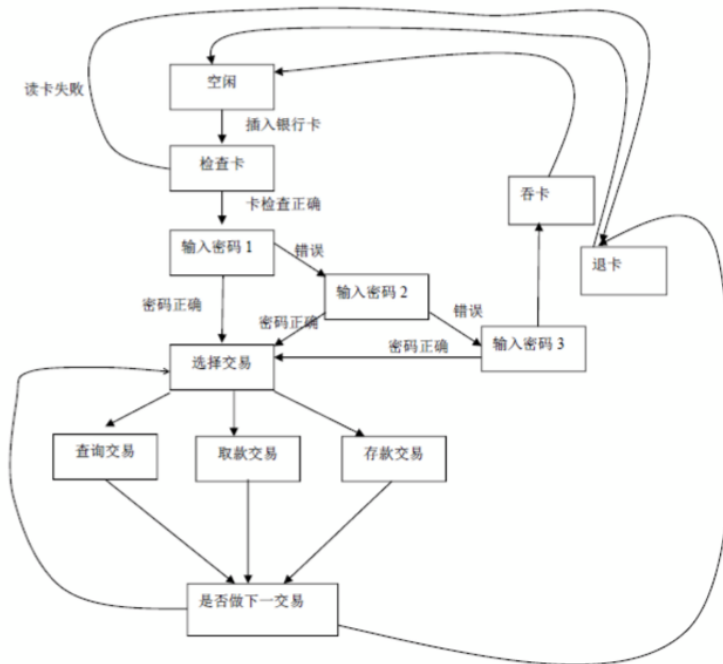
1. 状态图→进一步完善类的方法

状态图表示了每个类的活跃状态 STE

三要素(状态图)

状态图: $\left\{ \begin{array}{l} \text{state} \\ \text{transition} \\ \text{event} \end{array} \right.$ 从一个状态到另一个状态的转变/迁移
 event 一个事件发生

eg: ATM 状态图.



(以事件驱动的项目可能会有)

→ 整个系统的状态图

不一定是每个项目均有

system level

↓

为程序员设计提供依据

一个状态对应一个程序的
一个 case 分支

但若是用面向对象开发的, 则每个 entity class / business class 均有状态图
 在设计类时作为交叉方法的依据

ATM
 ↓ 插入银行卡

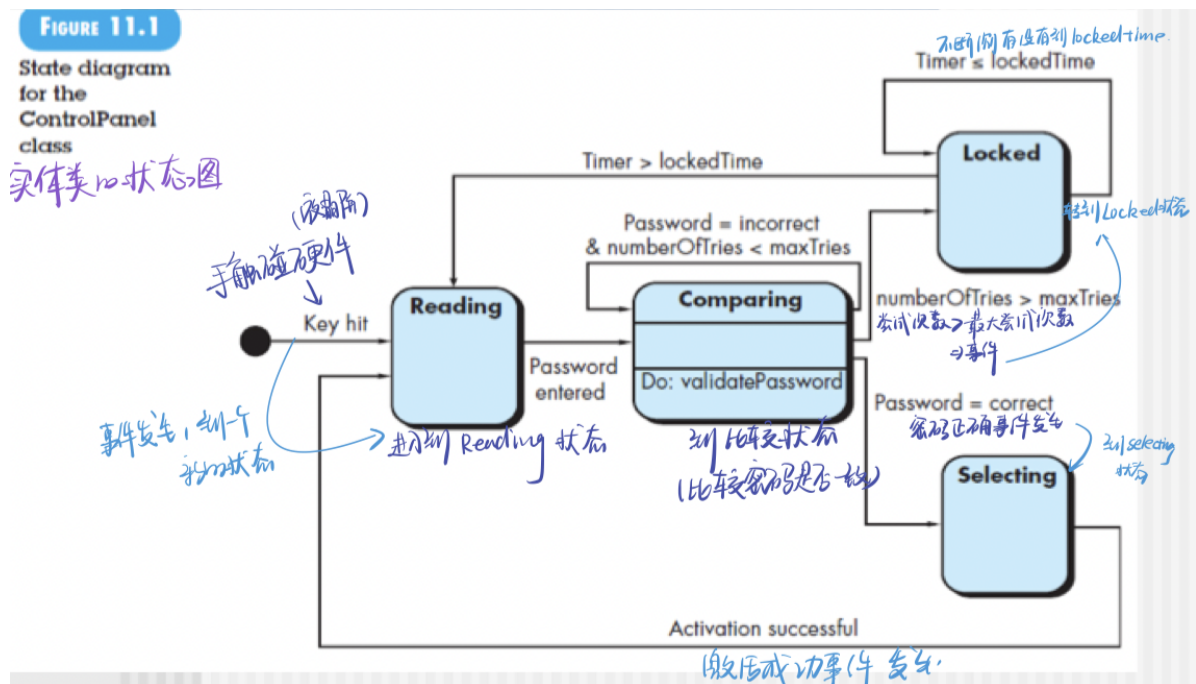
系统状态图一定有, 而在面向对象设计开发时每个类图也要有状态图

guard 保护功能: 一个布尔条件, 是状态转移发生时的判别式

【必考内容】

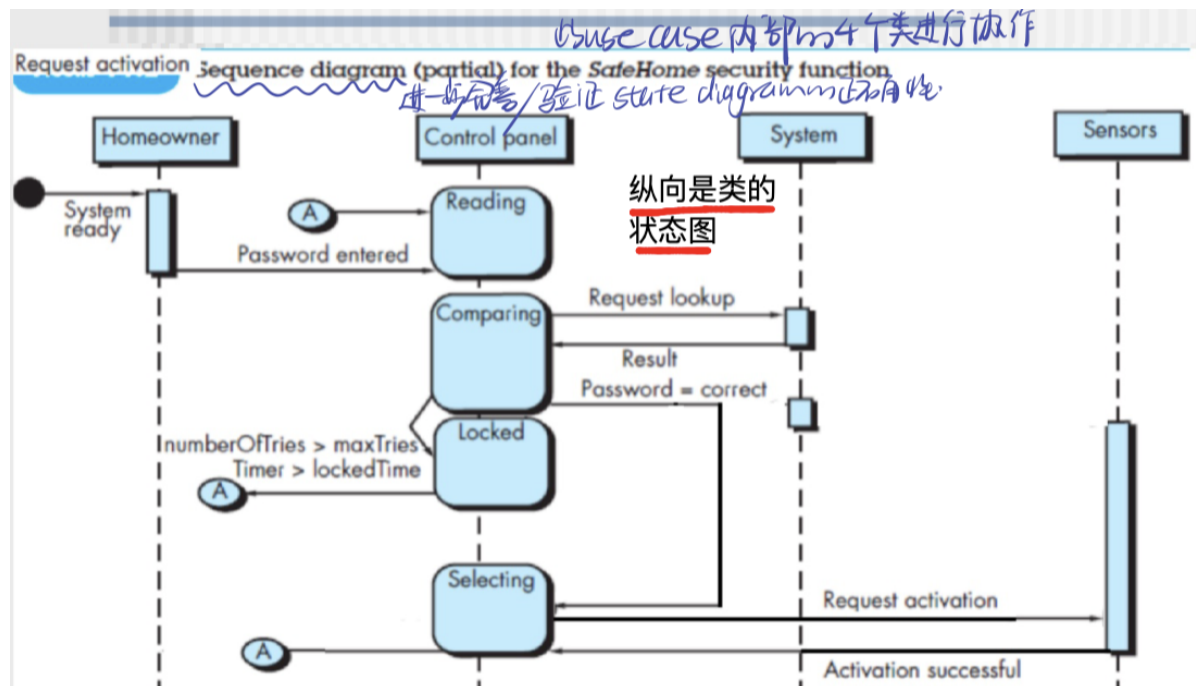
状态图中的方法不一定是类内的方法，也有可能是类间协作的方法

每一个state都有自己对应的方法



2. 时序图sequence→实例化的对象怎么交互，来完成UseCase的功能

时序图是用例的速写版本。它表示关键类和导致行为从类流到类的行为的事件



用例足以描述分析水平的相互作用（在设计期间引入了进一步的细化和细节），但时序图可以更加严格语法化规范化。

11.4 需求模型的模式Patterns for Requirement Modeling

分析模式Analysis Patterns存储在存储库中，以便软件团队的成员可以使用搜索工具来查找和重用它们。

11.5 Web和Mobile应用的需求建模

移动应用程序往往更专注并提供更有限的计算和操纵功能。无论功能性的广度如何，都需要分析功能要求，并在必要时建模。

11.5.1 分析到什么程度

取决于以下因素:

1. 应用增量的大小和复杂度
2. stakeholder的数量
3. 开发团队的规模
4. 开发团队有没有之前合作过的经验
5. 软件开发的成功对整个开发组织活动的成功的重要性

随着项目变小，利益相关者的数量较少，开发团队更具凝聚力，而且应用程序不太重要，应用更轻质的分析方法是合理的，这个时候analyse就没必要太细了

11.5.2 输入

不重要应该

11.5.3 前端需求建模的要求（输出）【重要】

前端模型的5个主要种类:

1. 控件/内容模型 **Content Model** 选择内容对象和内容对象的布局，比如文本图形图像视频等数据
 2. 交互模型 **Interaction Model** 通过交互完成功能
 3. 功能模型 **Functional Model** 定于用于内容对象的操作
- 【前三者是前端页面设计的重要依据】
4. 导航模型 **Navigation Model** 完成特定功能的导航（完成应用程序的总体导航策略）
 5. 配置模型 **Configuration Model** 配置图，比如前端、后端、网络等配置

以上5个部分确定后，可进行原型模型开发

11.5.4 内容模型 Content Model ①

内容模型包含结构元素structural elements，为应用程序提供了重要内容要求的重要视图

比如：HomeKeep系统中购买零部件界面中，包含信息的控件（图片，购买说明，宣传视频，价格栏.....），他们可能放在不同界面，所以要结构化

Content objects can be determined directly from use cases by examining the scenario description for direct and indirect references to content.

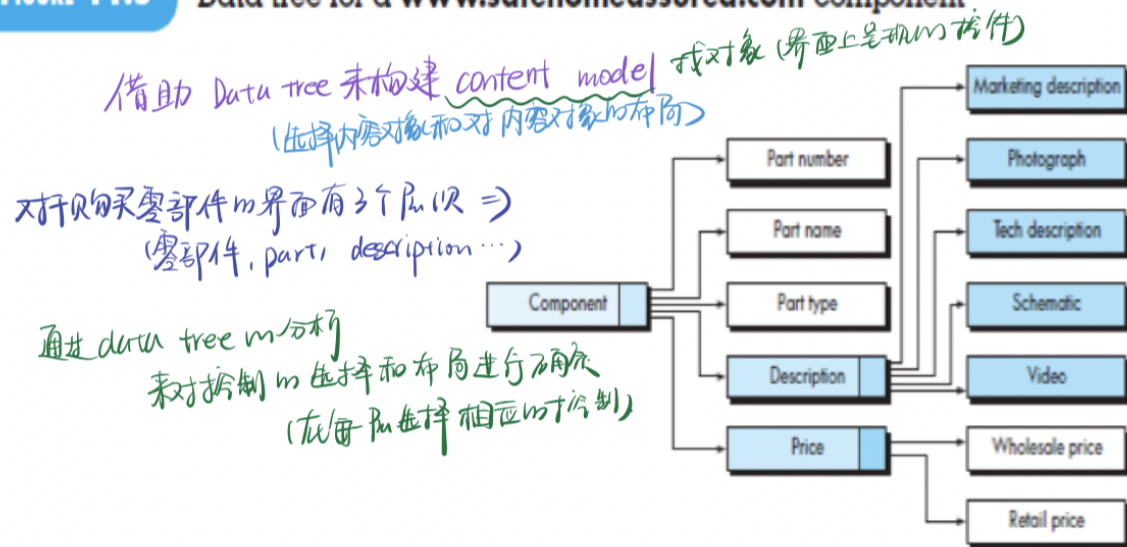
在许多情况下，与每个对象的简要描述耦合的内容对象的简单列表足以定义必须设计和实现的内容的要求。

※必考：data tree是干什么的

※表达方式：Data Tree（内容模型的数据结构：数据分解树）

FIGURE 11.5

Data tree for a www.safehomeassured.com component



11.5.5 交互建模 Interaction Model ②

核心：通过哪些页面交互来实现某种功能

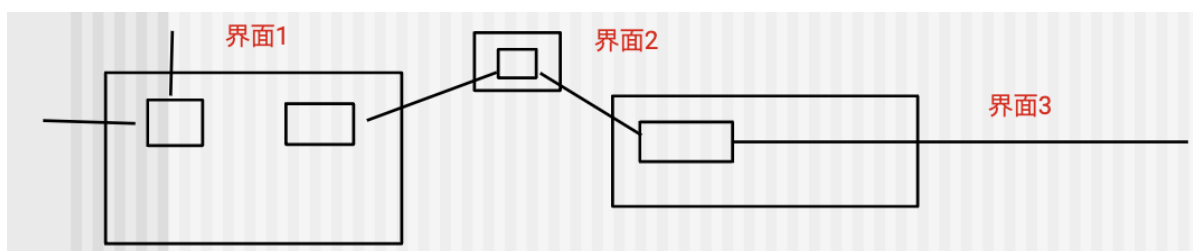
在内容建模的基础上，针对UseCase的功能，用界面将它的功能反应出来（借助Activity Diagram）来交互建模

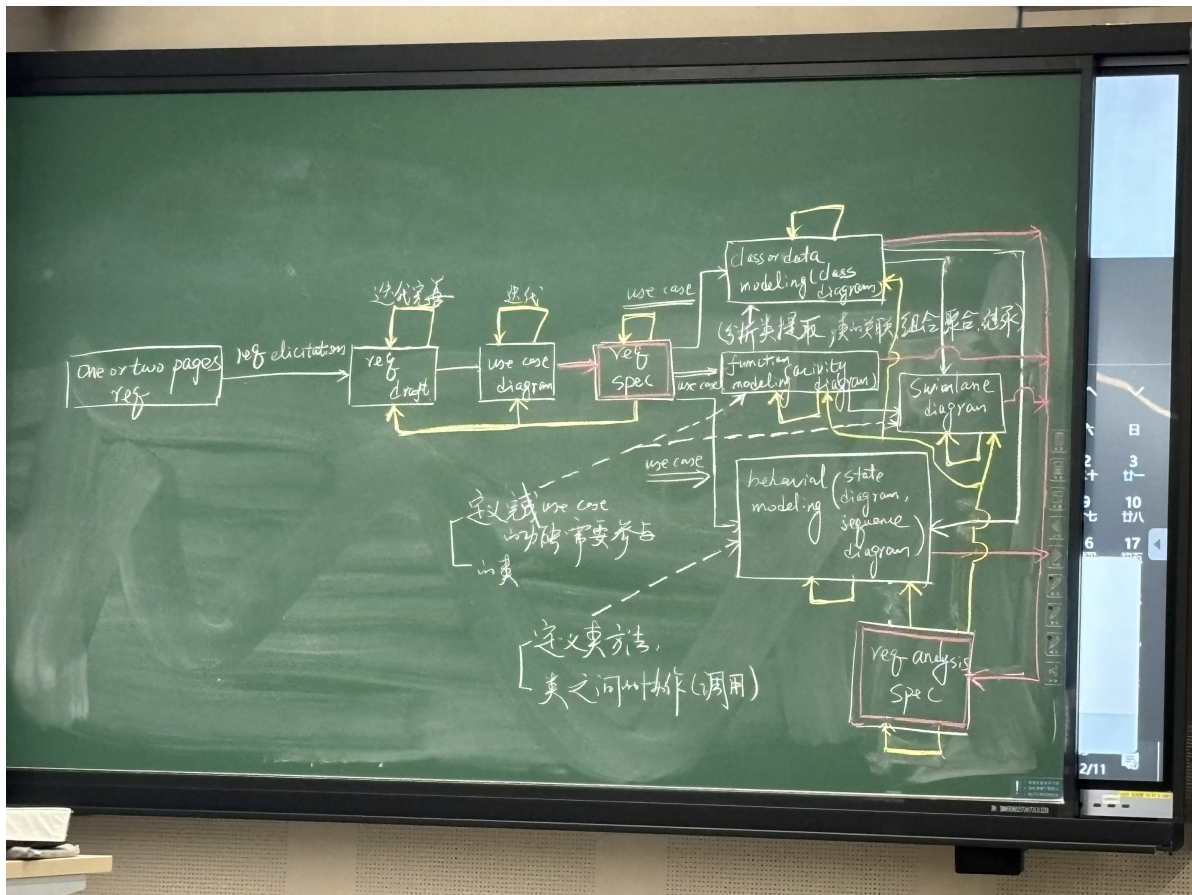
组成元素：（一个或多个）

- 用例图U
- 序列图S
- 状态图S
- 用户建模原型prototypes P

虽然可以说，创建用户界面原型是一种设计活动，但在创建分析模型Analysis Model期间执行它是一个好主意。因为用户界面prototype做完了，则有很多部分得到了确定

例子：





练习：创建接口原型的工具（Azure）

11.5.6 功能建模Functional Model ③

在交互界面的基础上，借助之前的activity diagram，对界面进行细化

功能模型解决了两个应用程序处理元素，其中每个应用程序要素表示不同的过程抽象级别：

1. 由应用程序传递给最终用户的用户可观察功能 user observable functionality
2. 在实现与之相关联的行为的分析类中包含的操作类。

关键：在交互模型的基础上，借助activity diagram对界面功能逻辑进行细化。比如：通过退货的活动图，画出退货功能的流程图，然后找出完成这个功能所需要进行交互的内容对象

11.5.7 导航建模 Navigation Modeling ④

导航通常受到相对简单的按钮列表和基于图标的菜单的约束

建立导航建模的一些需要考虑的问题：（PPT32页，爱谁看谁看吧，反正记不住）

11.5.8 配置建模Configure Model

配置模型是服务器端和客户端属性的列表。把网络IP，交换机，服务器连接起来的普通图。是High-Level Design